DEVELOPER ONBOARDING · CA FIRM SAAS

Learning MERN by shipping a feature

A study checklist plus a hands-on build: you'll add a working "Announcements" feature to this codebase, touching every layer from MongoDB to the React sidebar. Written for someone coming from Angular.

---

1 · Run the project

DO THIS FIRST — EVERYTHING ELSE MAKES MORE SENSE ONCE IT'S ON SCREEN

Read this before anything else. Your machine defaults to Node 16, but this project needs Node 18+. Your Angular project needs the old one, so switch every time you change projects:

```
nvm use 22.22.0
```

If you ever see `crypto.getRandomValues is not a function`, this is why.

One-time setup

```
# from the ca-firm-saas folder
npm install

# create your local config from the template
cp server/.env.example server/.env
```

The defaults in `.env` already point at a local MongoDB (`mongodb://127.0.0.1:27017/ca-firm-saas`). Make sure MongoDB is running.

Every day

```
npm run dev
```

That one command starts *both* halves — the API on port 5000 and the React app on 5173. Open **http://localhost:5173**. The very first account you register becomes Super Admin.

Other commands worth knowing

COMMAND	WHAT IT DOES
`npm run test -w server`	Runs the backend test suite
`npm run build -w client`	Production build of the React app
`npm run set-password -w server -- <email> <pass>`	Force-set any account's password
`npm run seed -w server`	Seeds a starter firm + admin

Why one command starts two apps. The root `package.json` uses *npm workspaces* — `client` and `server` are separate packages inside one repo, and `concurrently` runs both dev servers. The `-w server` flag means "run this script in the server workspace".

2 · Angular → MERN translation

YOU ALREADY KNOW MOST OF THESE IDEAS UNDER DIFFERENT NAMES

ANGULAR	THIS PROJECT	NOTE
Component (`.ts` + `.html` + `.css`)	One `.jsx` file	Markup and logic live together
`*ngFor`	`{items.map(x => …)}`	Plain JavaScript, not a directive
`*ngIf`	`{cond && <Thing/>}`	Or a ternary
`[(ngModel)]`	`useState` + `value` / `onChange`	No two-way binding — you wire both directions
`ngOnInit()`	`useEffect(fn, [])`	Also covers ngOnChanges/ngOnDestroy
Injectable service + DI	A module you `import`	No DI container, no providers array
`HttpClient`	`axios`	`client/src/services/apiClient.js`
RxJS `Observable`	`Promise` / `async-await`	No `.subscribe()`, no operators
`@NgModule`	—	Nothing to declare; just import what you use
Router + guards	`react-router-dom` + `ProtectedRoute.jsx`	Guards are just components
TypeScript	Plain JavaScript	Deliberate choice for this project

The one real mental shift. In React, the UI is a *function of state*. You never reach out and update the DOM — you call a state setter and React re-renders that component. There's no change detection to reason about and no `NgZone`. If the screen isn't updating, you changed something React isn't tracking as state.

3 · The study checklist

TICK AS YOU GO — THE BOXES REMEMBER NOTHING, SO SCREENSHOT IF YOU CARE

01 Modern JavaScript

~1 week

Do not skip this. MERN is ~90% plain JavaScript; every framework difficulty you'll hit is actually a JS gap.

- [ ] Arrow functions, destructuring, spread/rest
  You'll see `{ ...prev, name }` in every form in this repo
- [ ] Promises & `async/await`, `try/catch`
  Every API call and DB query
- [ ] `map` / `filter` / `find` / `reduce`
  Rendering lists and shaping data
- [ ] Optional chaining `?.` and `??`
  Used constantly for possibly-missing API data
- [ ] Modules: `require/module.exports` vs `import/export`
  Server uses the first, client the second — in the same repo

02 Node & npm

~2 days

- [ ] `package.json`, scripts, dependencies vs devDependencies
- [ ] Environment variables + `dotenv`
  Read `server/src/config/env.js` — it's the whole config surface
- [ ] npm workspaces (a monorepo)
  Why `-w server` exists

03 Express — the backend

~1 week

The genuinely new piece for you. Everything else has an Angular analogue; this doesn't.

- [ ] Routing: `router.get/post/patch/delete`, URL params, query strings
- [ ] Middleware — the `(req, res, next)` chain
  This is the core Express idea. Auth, validation and error handling are all middleware
- [ ] Status codes: 200 / 201 / 400 / 401 / 403 / 404 / 422 / 500
- [ ] Centralised error handling
  See `asyncHandler.js` + `errorHandler.middleware.js`

04 MongoDB & Mongoose

~4 days

- [ ] Documents vs SQL rows; collections vs tables
- [ ] Mongoose schemas & models; field types, `enum`, `default`, `required`
- [ ] Queries: `find`, `findOne`, `findByIdAndUpdate`, `countDocuments`
- [ ] `ref` + `populate()`
  Mongo's version of a JOIN — how a task carries its client's name
- [ ] Install **MongoDB Compass** and browse your own data
  Fastest way to make the database feel real

05 React ~1.5 weeks

- [ ] JSX, components, props
- [ ] `useState` — and why you never mutate state directly
- [ ] `useEffect` and the dependency array
  The #1 source of infinite-loop bugs. Learn it properly
- [ ] Lists + the `key` prop; conditional rendering
- [ ] Controlled form inputs
- [ ] `useContext`
  How `AuthContext` shares the logged-in user app-wide
- [ ] Custom hooks
  Then read `hooks/useApiData.js` — you'll understand the whole app

06 The glue ~3 days

- [ ] `react-router-dom`: routes, nested routes, `<Outlet/>`, `useNavigate`
- [ ] axios + interceptors
  `apiClient.js` auto-refreshes expired tokens — read it twice
- [ ] Vite: dev server, the `/api` proxy, `npm run build`
- [ ] JWT auth: access vs refresh tokens, httpOnly cookies, bcrypt

Realistic total: 4–6 weeks part-time to comfortably edit this codebase. You can start the build below after phases 1–4.

---

4 · Guided tour of this repo

READ IN THIS ORDER — YOU ALREADY KNOW WHAT THE APP DOES, WHICH MAKES THE CODE TEACH FASTER THAN ANY TUTORIAL

Backend: follow one request all the way down

1. `server/src/server.js` → `app.js` → `routes/index.js` — how a request enters and gets dispatched
2. `server/src/features/clients/` — read the folder in this order: **routes** → **controller** → **service** → **repository** → **model**. Every feature follows this exact shape, so learning one teaches all of them
3. `server/src/middlewares/auth.middleware.js` and `rbac.middleware.js` — who you are, and what you're allowed to do
4. `server/src/repositories/BaseRepository.js` — the *only* place firm-scoping and soft delete are enforced
5. `server/src/utils/` — `ApiResponse.js`, `asyncHandler.js`, `AppError.js`: the three conventions every file relies on

What each backend layer is for

LAYER	JOB	NEVER DOES
routes	Maps URL + method → controller, attaches auth/validation	Business logic
controller	Reads `req`, calls a service, returns `ApiResponse`	Touch the database

service	The actual business rules	Know about HTTP
repository	Database queries, firm scoping, soft delete	Business rules
model	Schema & field validation	Anything else

Frontend

6. `client/src/App.jsx` — every route in one file; start here
7. `client/src/services/apiClient.js` — axios setup + the token-refresh interceptor
8. `client/src/features/auth/context/AuthContext.jsx` — global auth state
9. `client/src/hooks/useApiData.js` — the fetch hook nearly every page uses
10. `client/src/features/clients/pages/ClientsPage.jsx` — a typical page tying it all together

> **Also read** `PROJECT_STATE.md` in the repo root. It documents every feature, why it was built that way, and the bugs that shaped it.

---

5 · Build a feature end to end

ANNOUNCEMENTS — ADMINS POST A NOTICE, ALL STAFF SEE IT

This is small but touches *every* layer: a Mongo model, the four backend layers, role gating, an API client, a React page, a route, and a sidebar link. Roughly 45–90 minutes. Type the code rather than pasting — that's the point.

1

The model — what an announcement *is*

Create the folder `server/src/features/announcements/models/`, then:

▸ server/src/features/announcements/models/Announcement.model.js

```
const mongoose = require('mongoose');
const auditPlugin = require('../../../models/plugins/auditPlugin');

const announcementSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    body: { type: String, required: true, trim: true },
  },
  { timestamps: true }
);

// Adds firmId, createdBy, isDeleted, deletedAt — every tenant-owned
// collection uses this, which is what keeps firms isolated from each other.
announcementSchema.plugin(auditPlugin);

module.exports = mongoose.model('Announcement', announcementSchema);
```

Notice: you never declared `firmId` — the plugin adds it. That's the multi-tenancy backbone.

2 The repository — database access

▸ server/src/features/announcements/repositories/announcement.repository.js

```
const BaseRepository = require('../../../repositories/BaseRepository');
const Announcement = require('../models/Announcement.model');

class AnnouncementRepository extends BaseRepository {
  constructor() {
    super(Announcement);
  }
}

module.exports = new AnnouncementRepository();
```

That's genuinely all — `BaseRepository` already gives you `create`, `paginate`, `findById`, `updateById` and `softDelete`, each automatically scoped to the caller's firm.

3 The service — business rules

▸ server/src/features/announcements/services/announcement.service.js

```
const announcementRepository = require('../repositories/announcement.repository');
const { parseListQuery } = require('../../../utils/pagination');

async function listAnnouncements(firmId, query) {
  const { page, limit, skip, sort } = parseListQuery(query, {
    searchableFields: ['title'],
  });

  const { items, total } = await announcementRepository.paginate(firmId, {
    sort,
    skip,
    limit,
    populate: { path: 'createdBy', select: 'name' },
  });

  return { items, page, limit, total };
}

async function createAnnouncement(firmId, actorId, data) {
  return announcementRepository.create(firmId, {
    title: data.title.trim(),
    body: data.body.trim(),
    createdBy: actorId,
  });
}

module.exports = { listAnnouncements, createAnnouncement };
```

`populate` is the JOIN: it swaps the `createdBy` id for `{ name }` so the UI can show who posted it.

4 The controller — HTTP in, envelope out

▸ server/src/features/announcements/controllers/announcement.controller.js

```
const asyncHandler = require('../../../utils/asyncHandler');
const ApiResponse = require('../../../utils/ApiResponse');
const announcementService = require('../services/announcement.service');

const list = asyncHandler(async (req, res) => {
  const { items, page, limit, total } =
    await announcementService.listAnnouncements(req.user.firmId, req.query);
  return ApiResponse.paginated(res, { items, page, limit, total });
});

const create = asyncHandler(async (req, res) => {
  const announcement = await announcementService.createAnnouncement(
    req.user.firmId,
    req.user.id,
    req.body
  );
  return ApiResponse.created(res, {
    message: 'Announcement posted',
    data: { announcement },
  });
});

module.exports = { list, create };
```

`asyncHandler` means you never write `try/catch` here — any thrown error is forwarded to the global error handler automatically. `req.user` was attached by the auth middleware.

5 The routes — and the permission gate

▸ server/src/features/announcements/routes/announcement.routes.js

```
const { Router } = require('express');
const controller = require('../controllers/announcement.controller');
const { requireAuth } = require('../../../middlewares/auth.middleware');
const { requireAdminRole } = require('../../../middlewares/rbac.middleware');

const router = Router();

router.use(requireAuth);                      // everything below needs a login

router.get('/', controller.list);              // any signed-in staff member
router.post('/', requireAdminRole, controller.create);  // admins only

module.exports = router;
```

Now register it. In `server/src/routes/index.js`, add the import alongside the others and one line:

```
const announcementRoutes = require('../features/announcements/routes/announcement.routes');
// ...
router.use('/announcements', announcementRoutes);
```

Test the backend before touching React. Restart the server, then in the browser console while logged in:

```
await (await fetch('/api/v1/announcements')).json()
```

You should get `{ success: true, data: [], meta: {...} }`. Half of backend work is confirming each layer before adding the next.

6 The API client

▸ client/src/features/announcements/services/announcement.service.js

```
import apiClient from '../../../services/apiClient';

export const announcementApi = {
  list: (params) =>
    apiClient.get('/announcements', { params }).then((r) => r.data),
  create: (payload) =>
    apiClient.post('/announcements', payload).then((r) => r.data.data.announcement),
};
```

No tokens here — the interceptor in `apiClient.js` attaches them and retries after refreshing if one expired.

7 The page

- client/src/features/announcements/pages/AnnouncementsPage.jsx

```
import { useState } from 'react';
import { announcementApi } from '../services/announcement.service';
import useApiData from '../../../hooks/useApiData';
import { useAuth } from '../../auth/context/AuthContext';
import { useToast } from '../../../components/shared/Toast';
import { extractErrorMessage } from '../../../services/apiClient';
import { isAdminRole } from '../../../constants/roles';
import Card, { CardHeader } from '../../../components/ui/Card';
import FormField from '../../../components/ui/FormField';
import Input from '../../../components/ui/Input';
import Textarea from '../../../components/ui/Textarea';
import Button from '../../../components/ui/Button';
import Spinner from '../../../components/ui/Spinner';
import { formatDate } from '../../../utils/format';

export default function AnnouncementsPage() {
  const { user } = useAuth();
  const toast = useToast();
  const [form, setForm] = useState({ title: '', body: '' });
  const [saving, setSaving] = useState(false);

  const { data, loading, refetch } = useApiData(
    () => announcementApi.list({ limit: 20 }),
    []
  );
  const announcements = data?.data || [];

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSaving(true);
    try {
      await announcementApi.create(form);
      toast.success('Announcement posted');
      setForm({ title: '', body: '' });
      refetch();
    } catch (err) {
      toast.error(extractErrorMessage(err));
    } finally {
      setSaving(false);
    }
  };

  return (
    <div style={{ display: 'flex', flexDirection: 'column', gap: 16 }}>
      <h1 style={{ fontSize: 20, margin: 0 }}>Announcements</h1>

      {isAdminRole(user?.role) && (
        <Card>
          <CardHeader title="Post an announcement" subtitle="Everyone in the firm will see this" />
          <form onSubmit={handleSubmit}>
            <FormField label="Title" required>
              <Input
                value={form.title}
                onChange={(e) => setForm((p) => ({ ...p, title: e.target.value }))}
              />
            </FormField>
            <FormField label="Message" required>
              <Textarea
                rows={3}
                value={form.body}
                onChange={(e) => setForm((p) => ({ ...p, body: e.target.value }))}
              />
            </FormField>
            <Button type="submit" loading={saving}
```

```
              disabled={!form.title.trim() || !form.body.trim()}>
              Post
            </Button>
          </form>
        </Card>
      )}

      {loading ? (
        <Spinner label="Loading…" />
      ) : announcements.length === 0 ? (
        <Card><p style={{ margin: 0, color: 'var(--color-text-muted)' }}>
          No announcements yet.
        </p></Card>
      ) : (
        announcements.map((a) => (
          <Card key={a._id}>
            <CardHeader
              title={a.title}
              subtitle={`${a.createdBy?.name || 'Someone'} · ${formatDate(a.createdAt)}`}
            />
            <p style={{ margin: 0, whiteSpace: 'pre-line' }}>{a.body}</p>
          </Card>
        ))
      )}
    </div>
  );
}
```

Four React ideas are doing all the work here, and they're the same four you'll use forever:

- `useState` holds the form; `{ ...p, title }` replaces state rather than mutating it
- `useApiData` fetches on mount and hands back `refetch`
- `.map()` renders the list — note the required `key`
- `isAdminRole(...) && (...)` hides the form from non-admins *visually*; `requireAdminRole` on the route is what actually enforces it

8 Route it and link it

In `client/src/App.jsx`, add the lazy import next to the others:

```
const AnnouncementsPage = lazy(() => import('./features/announcements/pages/AnnouncementsPage'));
```

…then add the route inside the CRM `<ServiceRoute service="crm">` block:

```
<Route path="/crm/announcements" element={<AnnouncementsPage />} />
```

Finally, put it in the sidebar. In `client/src/constants/services.js`, add to the CRM menu's *Engagement* group (and import an icon like `Megaphone` from `lucide-react` at the top):

```
{ label: 'Announcements', path: '/crm/announcements', icon: Megaphone },
```

9 Prove it works

- [ ] Sidebar shows **Announcements** under CRM → Engagement
- [ ] As Super Admin you see the post form; posting adds a card immediately
- [ ] Refresh the page — it's still there (it's in MongoDB, not memory)
- [ ] Find it in Compass: database `ca-firm-saas`, collection `announcements`
- [ ] Log in as an **employee** — the list shows, the form does not
- [ ] As that employee, run this in the console — you should get **403**, proving the server enforces it independently of the UI:

```
await fetch('/api/v1/announcements', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: 'x', body: 'y' })
}).then(r => r.status)
```

That last check is the most important idea in the whole codebase: **hiding a button is UX, not security.** The server always decides.

Then extend it yourself

Now that the slice exists, these are small, high-learning additions:

- **Delete** (admin only) — add `softDelete` in the service, a `DELETE /:id` route, a button on the card
- **Validation** — write `validators/announcement.validator.js` following `document.validator.js`, and wire it with the `validate()` middleware
- **Pagination** — a "Load more" button using the `meta.totalPages` already being returned
- **Notify everyone** — call `notificationService.notify()` when one is posted (see `document.service.js` for the pattern)
- **Show it in the Client Portal** — the hard one, and a real test of whether the scoping rules clicked

---

6 · Every concept → a real file in this project

STUDYING A TOPIC? OPEN ITS FILE HERE STRAIGHT AFTER — THEORY AND CODEBASE IN ONE PASS

JavaScript & React

CONCEPT	OPEN THIS FILE	WHAT TO NOTICE
`useState` + spread updates	`client/src/features/clients/pages/ClientsPage.jsx`	Filters and modal state; `setForm(p => ({ ...p, name }))` never mutates
`useEffect` on mount	`client/src/features/auth/context/AuthContext.jsx`	The boot effect with `[]` deps — decides restore-vs-login once per load
Custom hook	`client/src/hooks/useApiData.js`	~45 lines that replace a whole data library; note the request-id counter that discards stale responses
`useContext`	`client/src/features/auth/context/AuthContext.jsx`	`useAuth()` at the bottom — every page gets the user without prop-passing
Controlled inputs	`client/src/features/settings/components/ChangePasswordCard.jsx`	Three inputs, one state object, inline validation before submit
List rendering + `key`	`client/src/components/ui/DataTable.jsx`	One generic table every page reuses via a `columns` array of render functions
Conditional render by role	`client/src/features/settings/pages/ProfileSettingsPage.jsx`	`{isAdminRole(user?.role) && <ChangePasswordCard />}`
Router + guards	`client/src/App.jsx` · `components/shared/ProtectedRoute.jsx`	Guards are just components rendering `<Navigate>` or `<Outlet>`

`useNavigate`	`client/src/components/shared/PanelAssistant.jsx`	The assistant's "Open Team" buttons navigate programmatically
axios + interceptors	`client/src/services/apiClient.js`	Token attach on request; silent refresh-and-retry on 401 — read twice

Node, Express & MongoDB

CONCEPT	OPEN THIS FILE	WHAT TO NOTICE
App boot & middleware order	`server/src/server.js` · `app.js`	Why models register first; why `app` and `listen` are split (tests)
Express routing	`server/src/features/clients/routes/client.routes.js`	Method + path + gate + controller, one line each, zero logic
Custom middleware	`server/src/middlewares/auth.middleware.js`	The `(req, res, next)` shape; attaches `req.user` from the JWT
Role/permission middleware	`server/src/middlewares/rbac.middleware.js`	`authorize('client:read')` — capability lists per role, one file
Body validation	`middlewares/validate.middleware.js` + `features/users/validators/user.validator.js`	Plain functions returning `{ isValid, errors, value }` — no library
Central error handling	`utils/asyncHandler.js` · `utils/AppError.js` · `middlewares/errorHandler.middleware.js`	Why no controller has try/catch; how Mongoose errors become clean 4xx
Mongoose schema	`server/src/features/clients/models/Client.model.js`	Field types, enums, nested objects, indexes
Schema plugin	`server/src/models/plugins/auditPlugin.js`	One plugin gives every collection `firmId` + soft delete
Queries & pagination	`repositories/BaseRepository.js` + `utils/pagination.js`	`scoped()` injecting `firmId`; `Promise.all` for items+count
`populate()` (the JOIN)	`server/src/features/documents/services/document.service.js`	`populate: { path: 'uploadedBy', select: 'name avatarUrl' }`
JWT sign/verify	`server/src/utils/jwt.js`	Access JWT vs opaque refresh token — different tools on purpose
bcrypt	`features/auth/services/auth.service.js` (login) · `utils/setPassword.js`	Hash on write, `compare` on login — plaintext is never stored
httpOnly cookies	`features/auth/controllers/auth.controller.js`	`refreshCookieOptions` — why JS can't read the refresh token
Env config	`server/src/config/env.js`	Whole config surface in one object; prod asserts required vars
Cron jobs	`server/src/jobs/index.js`	7 schedules, each wrapped so one failure can't kill the rest
File upload	`middlewares/upload.middleware.js` + `features/documents/routes`	multer memory storage → Cloudinary stream

How to use this table: when a checklist topic from section 3 feels abstract, open its file here and read it top to bottom — every file listed is under 150 lines. The tutorial teaches the pattern once; these show it used in production.

7 · When it breaks

THE ERRORS YOU WILL ACTUALLY HIT, AND WHAT THEY MEAN

SYMPTOM	USUAL CAUSE
`crypto.getRandomValues is not a function`	Node 16. Run `nvm use 22.22.0`
`MongooseServerSelectionError`	MongoDB isn't running, or `MONGO_URI` is wrong
Page blank, console shows a module error	Vite HMR cached a stale module — hard-refresh the browser
API returns `401` unexpectedly	Access token expired and refresh failed — log in again
API returns `403`	Working as designed: your role lacks that capability
API returns `422`	A validator rejected the body; the response lists exactly which field

"Cannot read properties of undefined"	Data hasn't loaded yet — guard with `data?.field`
Infinite re-render / endless network calls	`useEffect` dependency array — you're passing a new object or function each render
Route change does nothing	Route not registered in `App.jsx`, or a guard is bouncing you

How to debug each layer

- **Is it front or back?** Open DevTools → Network, click the failing request. If the response is wrong, it's the backend. If the response is right but the screen is wrong, it's React.
- **Backend:** plain `console.log` in the service — the server terminal prints it. Every request is logged there too.
- **Frontend:** install React DevTools; inspect a component's state and props directly.
- **Database:** Compass. If it's not in the collection, it never saved, no matter what the UI said.

> **The habit that matters most:** change one layer, verify it, then move to the next. Model → repository → service → controller → route → API client → page. When something breaks you'll know exactly which step did it, instead of debugging seven layers at once.

CA Firm SaaS — developer onboarding. Companion docs in the repo: `README.md` (setup) and `PROJECT_STATE.md` (every feature and why it exists).